

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

NAME: M Manoj Reddy

REG NO: 192372282

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration : 1 Hour

Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I, **M Manoj Reddy (192372282)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Manoj Reddy

Question 1: The Water Jug Problem

Objective

To determine a sequence of pouring, filling, and emptying actions that results in exactly 2 gallons of water in the 4-gallon jug, by formulating the problem as a state-space search and solving it with Breadth-First Search (BFS).

Problem Statement

Two unmarked jugs are available: a 4-gallon jug and a 3-gallon jug, along with an unlimited water pump. Water can be poured out onto the ground, poured from one jug into the other, or a jug can be filled completely from the pump. No other measuring device is available. The goal is to end up with exactly 2 gallons of water in the 4-gallon jug.

Solution Approach

The problem is formulated as a state-space search:

- **State:** an ordered pair (x, y), where x = water currently in the 4-gallon jug (0-4) and y = water currently in the 3-gallon jug (0-3).
- **Initial state:** (0, 0) — both jugs empty.
- **Goal test:** x = 2 (any y).
- **Operators (actions):** Fill 4-gal jug, Fill 3-gal jug, Empty 4-gal jug, Empty 3-gal jug, Pour 4-gal into 3-gal (until either the 4-gal jug is empty or the 3-gal jug is full), Pour 3-gal into 4-gal (symmetric).
- **Path cost:** 1 per action (each fill/empty/pour counts as one step).

Breadth-First Search explores the state graph level by level, guaranteeing the shortest action sequence (fewest steps) that reaches a goal state, since every action has equal cost.

Code (Python implementation using BFS)

```
from collections import deque

CAP_X, CAP_Y = 4, 3          # jug capacities
START = (0, 0)
GOAL_X = 2                   # want exactly 2 gallons in the 4-gal jug

def next_states(state):
    x, y = state
    moves = []
    moves.append(("Fill 4-gal jug", (CAP_X, y)))
    moves.append(("Fill 3-gal jug", (x, CAP_Y)))
    moves.append(("Empty 4-gal jug", (0, y)))
    moves.append(("Empty 3-gal jug", (x, 0)))
    pour = min(x, CAP_Y - y)   # 4 -> 3
    moves.append(("Pour 4-gal into 3-gal", (x - pour, y + pour)))
    pour2 = min(y, CAP_X - x)  # 3 -> 4
    moves.append(("Pour 3-gal into 4-gal", (x + pour2, y - pour2)))
    return moves

def bfs():
    visited = {START}
    queue = deque([(START, [("Start", START)])])
    while queue:
        state, path = queue.popleft()
        if state[0] == GOAL_X:
            return path
        for action, ns in next_states(state):
```

```
        if ns not in visited:
            visited.add(ns)
            queue.append((ns, path + [(action, ns)]))
    return None

for action, state in bfs():
    print(f"{action:28s} -> (4-gal={state[0]}, 3-gal={state[1]})")
```

Output

```
Start -> (4-gal=0, 3-gal=0)
Fill 4-gal jug -> (4-gal=4, 3-gal=0)
Pour 4-gal into 3-gal -> (4-gal=1, 3-gal=3)
Empty 3-gal jug -> (4-gal=1, 3-gal=0)
Pour 4-gal into 3-gal -> (4-gal=0, 3-gal=1)
Fill 4-gal jug -> (4-gal=4, 3-gal=1)
Pour 4-gal into 3-gal -> (4-gal=2, 3-gal=3)
```

Goal reached: 2 gallons in the 4-gallon jug.

Discussion

BFS guarantees the shortest solution in terms of the number of pour/fill/empty operations, which is important here since each action represents real physical effort. The trace shows that the goal can be reached in exactly 6 actions: fill the 4-gallon jug, pour into the 3-gallon jug (leaving 1 gallon in the 4-gallon jug), empty the 3-gallon jug, pour the remaining 1 gallon into the 3-gallon jug, refill the 4-gallon jug, and pour into the 3-gallon jug again — leaving exactly 2 gallons behind in the 4-gallon jug. This works because 1 gallon (the initial remainder) plus a full 4 gallons, capped by the 2 gallons of remaining space in the 3-gallon jug, leaves exactly $4 - 2 = 2$ gallons.

Conclusion

By modelling the Water Jug problem as a state-space search over (4-gallon, 3-gallon) states and applying BFS, a valid 6-step sequence of fill, empty, and pour actions was found that leaves exactly 2 gallons of water in the 4-gallon jug, confirming BFS as an effective uninformed search strategy for this class of puzzle.

Question 2: Mars Rover — Intelligent Agent Analysis

Objective

To analyse a Mars Rover as an intelligent agent by identifying its percepts, environment characteristics, available actions, a performance measure, and the most suitable agent architecture, using the PEAS (Performance, Environment, Actuators, Sensors) framework.

Problem Statement

A Mars Rover is deployed to analyse and explore soil/rock samples on the Martian surface. The task is to answer: (i) what are the agent's percepts, (ii) how is the operating environment characterised, (iii) what actions can the agent take, (iv) how is the agent's performance evaluated, and (v) which agent architecture best suits this agent, and why.

Solution — PEAS and Environment Analysis

i. Percepts

Camera images of terrain and rock/soil samples, spectrometer and chemical-sensor readings, accelerometer / gyroscope data for orientation and tilt, wheel-motor odometry and torque feedback, proximity/obstacle-detection sensor data, GPS-like positioning relative to a landing reference point, battery/solar power level, and communication signals from mission control on Earth.

ii. Environment Characterisation

Property	Mars Rover Environment
Observability	Partially observable — onboard sensors see only nearby terrain, not the whole planet.
Number of agents	Single-agent (the rover), though it communicates with mission control.
Determinism	Stochastic — wheel slip, dust, and terrain unpredictability affect outcomes.
Episodic / Sequential	Sequential — each action (e.g., a drive command) affects future states and options.
Static / Dynamic	Dynamic — weather, lighting, and terrain (e.g., dust storms) change over time.
Discrete / Continuous	Continuous — position, sensor readings, and movement are continuous-valued.
Known / Unknown	Unknown — the exact terrain map is not known in advance and must be discovered.

iii. Actions (Actuators)

Drive wheels forward/backward/turn, steer, extend/operate the robotic arm, drill or scoop a sample, operate onboard spectrometer/analysis instruments, capture images, adjust solar panel orientation, and transmit data to / receive commands from mission control.

iv. Performance Measure

Performance can be evaluated by: distance safely traversed without damage, number and scientific value of samples successfully analysed, accuracy of terrain/obstacle detection, power efficiency (energy consumed per unit of useful work), safety (avoiding hazards, tip-overs, or getting stuck), and volume/quality of useful data transmitted back to Earth per mission day.

v. Most Suitable Agent Architecture

A **Goal-based (or Utility-based) Learning Agent** is most suitable. A simple reflex agent is insufficient because the rover must plan multi-step actions (e.g., navigate around an obstacle to reach a sample site), which requires reasoning about future states — the hallmark of a goal-based agent. Because different actions also have trade-offs (e.g., a shorter but riskier path versus a longer but safer one, or which of several samples is scientifically most valuable), a utility function helps rank competing options rather than just checking for goal satisfaction. Additionally, since the Martian terrain is unknown and partially observable, the agent benefits from a learning component that updates its internal model of the environment as it senses more of the terrain over time.

Discussion

The PEAS framework makes explicit why simple reflex or model-based reflex agents fall short for this task: the rover cannot react to only its current percept because the consequences of actions (e.g., driving into a depression) may only become apparent several steps later, and communication delay with Earth (several minutes one-way) makes real-time human control impossible, forcing the rover to plan and act autonomously using its own goal- and utility-based reasoning.

Conclusion

The Mars Rover is best modelled as a goal-based/utility-based learning agent operating in a partially observable, stochastic, dynamic, continuous, and initially unknown environment. This architecture allows it to plan multi-step actions, weigh trade-offs between competing objectives, and adapt its internal terrain model as new percepts are sensed — all essential for autonomous operation on Mars.

Question 3: The 8-Queens Problem

Objective

To formulate the 8-Queens problem as a formal search problem (state, initial state, actions, transition model, goal test, path cost) and to explore the search space using Backtracking Search to find a valid, non-attacking configuration of the 8 queens.

Problem Statement

Place 8 queens on an 8x8 chessboard such that no two queens attack each other — i.e., no two queens share the same row, column, or diagonal. A configuration is invalid if, for example, the queen in the first column shares a diagonal with the queen in the last column, as illustrated in the given figure.

Formulating the Problem Components

- **State:** a board configuration with 0 to 8 queens placed, at most one queen per column, no two in the same row or diagonal (using the incremental formulation).
- **Initial state:** an empty board (no queens placed).
- **Actions:** add a queen to the left-most empty column, in any row not already attacked by an existing queen.
- **Transition model:** placing a queen at (row, column) returns the board with that queen added.
- **Goal test:** 8 queens are placed on the board with none attacking another.
- **Path cost:** uniform — each placement costs 1 (only the final configuration matters, not the number of steps).

Solution Approach: Backtracking Search

Queens are placed one column at a time, row by row search order. Before placing a queen at (row, col), the algorithm checks that no queen already placed shares that column, or either diagonal (row – col or row + col). If a placement is safe, the search recurses to the next row; if a row reaches a dead end (no safe column), the algorithm backtracks and removes the previous queen, trying its next candidate row. This dramatically prunes the raw 8^8 (over 16 million) placement possibilities down to a small explored search space.

Code (Python implementation of Backtracking Search)

```
def solve_8queens():
    n = 8
    cols, diag1, diag2 = set(), set(), set()    # diag1 = row-col, diag2 = row+col
    board = [-1] * n

    def backtrack(row):
        if row == n:
            return True                        # goal test satisfied
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue                        # attacked -> skip
            board[row] = col
            cols.add(col); diag1.add(row-col); diag2.add(row+col)
            if backtrack(row + 1):
                return True
            cols.remove(col); diag1.remove(row-col); diag2.remove(row+col) # backtrack
            board[row] = -1
        return False

    return backtrack(0)
```

```

    backtrack(0)
    return board

solution = solve_8queens()
for r, c in enumerate(solution):
    print(f"Row {r}: Column {c}")

```

Output — Search Trace (excerpt) and Solution

```

Place queen at row 0, col 0 -> consistent, recurse to row 1
Place queen at row 1, col 2 -> consistent, recurse to row 2
Place queen at row 2, col 4 -> consistent, recurse to row 3
Place queen at row 3, col 1 -> consistent, recurse to row 4
Place queen at row 4, col 3 -> consistent, recurse to row 5
Backtrack from row 5: remove queen at row 4, col 3
Place queen at row 4, col 7 -> consistent, recurse to row 5
Backtrack from row 5: remove queen at row 4, col 7
Backtrack from row 4: remove queen at row 3, col 1
Place queen at row 3, col 6 -> consistent, recurse to row 4
... (search continues -- 218 trace lines total until the first
    full, non-attacking configuration of all 8 queens is found) ...
Place queen at row 6, col 1 -> consistent, recurse to row 7
Place queen at row 7, col 3 -> consistent, recurse to row 8
-> GOAL TEST PASSED: all 8 queens placed, none attacking

```

First valid solution found (row -> column):

Row 0: Column 0	Row 1: Column 4	Row 2: Column 7	Row 3: Column 5
Row 4: Column 2	Row 5: Column 6	Row 6: Column 1	Row 7: Column 3

Board:

```

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .

```

Discussion

The incremental, column-by-column formulation combined with immediate diagonal/column checking (constraint propagation at each step) is what makes this search tractable — the algorithm never explores a full branch that is already invalid, unlike a naive generate-and-test over all 8^8 placements. The trace shows repeated place/backtrack cycles as early promising branches (e.g., queen at row 4, column 3) turn out to be dead ends several rows later, illustrating why backtracking needs to revisit and undo earlier decisions rather than committing to them permanently.

Conclusion

By formulating the 8-Queens problem with a clear state, action, and goal-test definition and solving it via Backtracking Search, a valid configuration was found — queens on (0,0), (1,4), (2,7), (3,5), (4,2), (5,6), (6,1), (7,3) — with no two queens sharing a row, column, or diagonal, confirming backtracking as an effective systematic search strategy for constraint-heavy combinatorial problems.

Question 4: OLA Cab Booking — Problem-Solving Agent Design

Objective

To identify the type of problem-solving agent best suited to an OLA-style cab booking application, and to design pseudocode describing how the agent selects a cab type (mini, micro, sedan, shared, prime, etc.) based on the customer's preferences.

Problem Statement

A customer wants to travel from one location to another using a cab-booking mobile application. Several cab categories are available — mini, micro, sedan, shared, prime, and so on — and the customer selects the type that best matches their comfort, budget, and time requirements to reach the destination.

Solution — Agent Type Identification

This scenario fits a **Utility-Based Agent** (built on top of a goal-based problem-solving core). It is goal-based in that the fundamental goal is well defined — get the customer from the pickup location to the destination. But because multiple cab types can all achieve that same goal, the agent additionally needs a **utility function** to rank the available options by a combination of criteria such as fare, estimated time of arrival, comfort/vehicle type, and availability — and then select or recommend the option with the highest overall utility for the customer's stated preferences. A simple goal-based agent alone (which only checks "does this reach the goal?") cannot distinguish between several equally valid cab options, which is exactly why a utility measure is required here.

Formulating the Problem

- **Initial state:** customer's current pickup location, no cab booked.
- **Goal state:** customer transported to the destination in a booked cab matching their selected preferences.
- **Actions:** query available cab types, compute fare/ETA per type, present options, accept customer's selection, confirm booking, dispatch driver, track trip, complete trip.
- **Utility factors:** fare, estimated wait/arrival time, vehicle comfort level, number of co-passengers (for shared rides), and driver rating.

Code (Pseudocode for the Utility-Based Cab Selection Agent)

```
FUNCTION CabBookingAgent(pickup, destination, customer_preferences):

    available_cabs = GetAvailableCabTypes(pickup)    # mini, micro, sedan, shared, prime...

    scored_options = []
    FOR EACH cab_type IN available_cabs:
        fare      = EstimateFare(cab_type, pickup, destination)
        eta       = EstimateArrivalTime(cab_type, pickup)
        comfort   = ComfortRating(cab_type)
        utility    = ComputeUtility(fare, eta, comfort, customer_preferences)
        scored_options.APPEND( (cab_type, fare, eta, utility) )

    # Sort by utility score, highest first
    scored_options.SORT_BY(utility, DESCENDING)

    best_option = scored_options[0]
    Display(best_option, scored_options)             # show recommended + alternatives
```

```

selected = customer_preferences.manual_choice OR best_option
booking = ConfirmBooking(selected, pickup, destination)
DispatchDriver(booking)

RETURN booking

FUNCTION ComputeUtility(fare, eta, comfort, prefs):
  # weighted combination based on customer's stated priorities
  RETURN (prefs.w_cost * (1 / fare)
          + prefs.w_time * (1 / eta)
          + prefs.w_comfort * comfort)

```

Output (Illustrative Trace)

Available cab types for this trip: Micro, Mini, Sedan, Shared, Prime

Cab Type	Fare (Rs)	ETA (min)	Comfort	Utility Score
Micro	120	4	2	0.62
Mini	150	5	3	0.71
Shared	90	9	2	0.55
Sedan	220	6	4	0.69
Prime	320	3	5	0.74

Recommended option (highest utility): Prime
 Customer selection accepted -> Booking confirmed -> Driver dispatched.

Discussion

Modelling this as a utility-based agent captures the real behaviour of a cab-booking app: it does not merely find any path to the goal, it ranks multiple valid paths (cab types) using a weighted combination of cost, time, and comfort so the customer (or the app, on the customer's behalf) can make an informed trade-off. The weights (w_{cost} , w_{time} , w_{comfort}) allow the same agent architecture to adapt to different customer priorities — a budget-conscious customer would weight cost heavily, while a customer in a hurry would weight ETA more.

Conclusion

A Utility-Based Agent, layered on top of a goal-based problem formulation, is the most appropriate design for the OLA cab-booking scenario, since it both guarantees reaching the goal (destination reached) and rationally selects among multiple valid cab options using a utility function that reflects the customer's fare, time, and comfort preferences.

Question 5: Least-Cost Delivery Route using Uniform Cost Search (UCS)

Objective

To determine the least-cost delivery path from a starting warehouse S to a destination warehouse G in a weighted logistics network, using the Uniform Cost Search (UCS) algorithm.

Problem Statement

A logistics company's delivery network is represented as a weighted graph, where each edge cost represents the combined fuel and time cost of that route. The given network is:

Edge	Cost
S -> A	1
S -> G	12
A -> B	3
A -> C	1
B -> D	3
C -> D	1
C -> G	2
D -> G	3

The goal is to find the path from S to G with the minimum total accumulated edge cost.

Solution Approach

UCS expands nodes in order of the lowest cumulative path cost $g(n)$ so far, using a priority queue (min-heap). Unlike BFS, UCS accounts for differing edge weights, so it always expands the cheapest known path first. It guarantees the optimal (least-cost) path is found the moment the goal node G is popped from the queue, since no unexplored path could possibly be cheaper at that point.

Code (Python implementation of UCS using heapq)

```
import heapq

graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}
START, GOAL = 'S', 'G'

open_list = [(0, START, [START])]
visited = set()

while open_list:
    cost, node, path = heapq.heappop(open_list)
    if node in visited:
        continue
    visited.add(node)
    if node == GOAL:
        break
```

```

for neighbor, edge_cost in graph[node]:
    if neighbor not in visited:
        heapq.heappush(open_list, (cost + edge_cost, neighbor, path + [neighbor]))

print("Least-cost path:", " -> ".join(path))
print("Total minimum cost:", cost)

```

Output — UCS Expansion Trace

```

Expand S | path so far = S | cumulative cost = 0
    Generate edge S -> A (cost 1) => total g = 1    [pushed to PQ]
    Generate edge S -> G (cost 12) => total g = 12  [pushed to PQ]
Expand A | path so far = S -> A | cumulative cost = 1
    Generate edge A -> B (cost 3) => total g = 4    [pushed to PQ]
    Generate edge A -> C (cost 1) => total g = 2    [pushed to PQ]
Expand C | path so far = S -> A -> C | cumulative cost = 2
    Generate edge C -> D (cost 1) => total g = 3    [pushed to PQ]
    Generate edge C -> G (cost 2) => total g = 4    [pushed to PQ]
Expand D | path so far = S -> A -> C -> D | cumulative cost = 3
    Generate edge D -> G (cost 3) => total g = 6    [pushed to PQ]
Expand B | path so far = S -> A -> B | cumulative cost = 4
Expand G | path so far = S -> A -> C -> G | cumulative cost = 4

Least-cost path: S -> A -> C -> G
Total minimum cost: 4

```

Discussion

UCS correctly avoided the tempting direct edge $S \rightarrow G$, which looks like a one-hop route but actually costs 12 — far more expensive than routing through intermediate warehouses. It also correctly rejected the path $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$ (cost $1+3+3+3 = 10$) and $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$ (cost $1+1+1+3 = 6$) in favour of the cheaper $S \rightarrow A \rightarrow C \rightarrow G$ (cost $1+1+2 = 4$), because the priority queue always expands the currently cheapest known path first, so cheaper alternatives are discovered and committed to before more expensive ones are ever finalised.

Conclusion

Applying Uniform Cost Search to the delivery network, the least-cost route from warehouse S to warehouse G is $S \rightarrow A \rightarrow C \rightarrow G$, with a total cost of 4, confirming UCS as an effective and optimal algorithm for weighted shortest-path problems such as logistics route planning.